

Calcolatori Elettronici

19 luglio 2023

Teoria

Esercizio 1 (12 punti): Realizzare la rete sequenziale che riconosca in input tutte le sequenze, anche sovrapposte, della parola `abba`, considerando come alfabeto di input $I = \{a, b, c, d\}$ e come alfabeto di output "stringa riconosciuta" e "stringa non riconosciuta". Dovendo riconoscere occorrenze sovrapposte, nel caso della seguente sequenza in input: `cbabbabba`, la macchina restituirà due volte l'output "stringa riconosciuta".

Si realizzi l'automa a stati finiti e si studi l'equazione di eccitazione di un flip flop di stato a scelta.

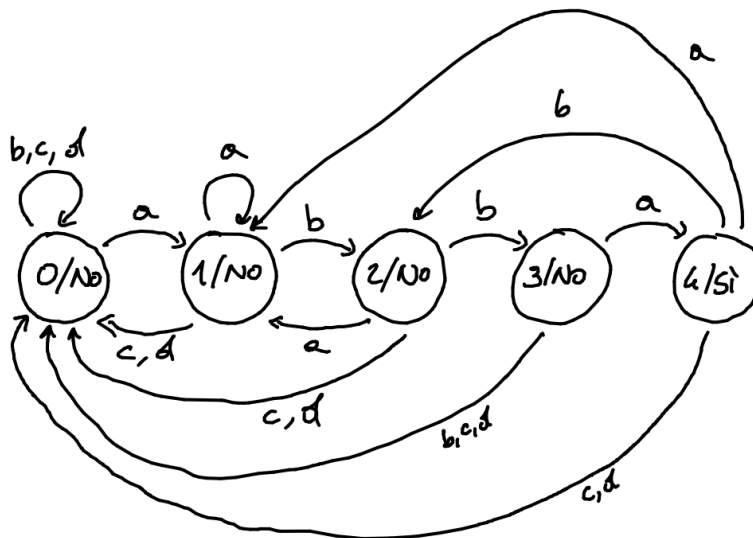
Esercizio 2 (12 punti): Codificare con codice di Hamming la parola binaria `0110`.

Esercizio 3 (6 punti): Descrivere il funzionamento di un half adder.

Soluzioni

Esercizio 1

Si propone una soluzione con l'utilizzo di una macchina di Moore. L'automa a stati finiti è il seguente:



Si utilizzano le seguenti codifiche degli alfabeti di input e di output (per semplicità, si indica con SÌ l'output "stringa riconosciuta" e con NO "stringa non riconosciuta"):

I	x_1	x_0		O	z_0
a	0	0		SÌ	1
b	0	1		NO	0
c	1	0			
d	1	1			

Poiché nell'automa sono presenti 5 stati, è necessario utilizzare $\lceil \log_2 5 \rceil = 3$ bit di stato. $2^3 - 5 = 3$ configurazioni di stato non sono ammissibili. La codifica degli stati utilizza la rappresentazione in base due del numero dello stato riportato nell'automa qui sopra. Si ottiene quindi la seguente tabella degli stati e transizioni (le configurazioni relative agli stati di partenza 101, 110, 111 non vengono mostrate, poiché non ammissibili—daranno adito a d.c.c. nelle mappe di Karnaugh):

x_1	x_0	y_2	y_1	y_0	y'_2	y'_1	y'_0
0	0	0	0	0	0	0	1
0	0	0	0	1	0	0	1
0	0	0	1	0	0	0	1
0	0	0	1	1	1	0	0
0	0	1	0	0	0	0	1
0	1	0	0	0	0	0	0
0	1	0	0	1	0	1	0
0	1	0	1	0	0	1	1
0	1	0	1	1	0	0	0
0	1	1	0	0	0	1	0
1	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0
1	0	0	1	0	0	0	0
1	0	0	1	1	0	0	0
1	0	1	0	1	0	0	0
1	1	0	0	0	0	0	0
1	1	0	0	1	0	0	0
1	1	0	1	0	0	0	0
1	1	0	1	1	0	0	0
1	1	1	0	0	0	0	0

Si sceglie di studiare l'equazione di eccitazione del flip flop di stato y_2 . La mappa di Karnaugh a 5 variabili è la seguente (non sono riportati gli zeri):

$$x_0 \ y_2$$

$$y_1 \ y_0$$

	00	01	11	10
00				
01		—	—	
11	1	—	—	
10		—	—	

$$x_4 = 0$$

$$x_0 \ y_2$$

$$y_1 \ y_0$$

	00	01	11	10
00				
01		—	—	
11		—	—	
10		—	—	

$$x_5 = 1$$

Pertanto, l'equazione di eccitazione del flip flop sarà $y_2' = \overline{x_0} \cdot \overline{x_1} \cdot y_1 \cdot y_0$.

Esercizio 2

Sono necessari tre bit di parità. Infatti, $m + 1 \leq 2^p - p$ (con $m = 4$ poiché si vuole codificare una parola di 4 bit) è soddisfatta per $p = 3$. I bit di parità vanno nelle posizioni che sono potenze di due. Si può effettuare il calcolo utilizzando la codifica tabellare, dove il simbolo ✓ indica le posizioni coperte da ciascun bit di parità.

	p_1	p_2	0	p_4	1	1	0	
p_1	✓		✓		✓		✓	1
p_2		✓	✓			✓	✓	1
p_4				✓	✓	✓	✓	0

Si ottiene quindi la codifica 1100110.

Progetto

Un processore **z64** gestisce il sistema di controllo di un condizionatore d'aria. Mediante la periferica sincrona **TERMOMETRO** è in grado di leggere la temperatura corrente della stanza, rappresentata come byte senza segno.

L'utilizzatore del condizionatore ha a disposizione un telecomando ad infrarossi che consente di impostare la temperatura desiderata e la modalità operativa (ventilatore, deumidificatore, condizionatore). Alla pressione del tasto di conferma, il telecomando trasmette la configurazione desiderata ad una periferica **RICEVITORE** che avverte in modalità asincrona lo **z64** del cambio di impostazione richiesto.

La periferica **RICEVITORE** ha a disposizione un registro di interfaccia tramite cui è possibile recuperare la configurazione richiesta. Tale configurazione è una parola di 8 bit, così composta:

- Il bit 7 (più significativo) è un flag booleano che indica che si vuole attivare la modalità ventilatore;
- Il bit 6 è un flag booleano che indica se si vuole attivare la modalità deumidificatore;
- Il bit 5 è un flag booleano che indica se si vuole attivare la modalità condizionatore;
- I 5 bit meno significativi rappresentano un intero senza segno che indica la temperatura richiesta.

Ad esempio, questa configurazione indica l'accensione del deumidificatore per raggiungere una temperatura di 23°C :

7	6	5	4	3	2	1	0
0	1	0	1	0	1	1	1

Alla ricezione della richiesta di interruzione da parte di **RICEVITORE**, il processore **z64** recupera la configurazione richiesta ed attiva/disattiva le periferiche sincrone **VENTILATORE**, **DEUMIDIFICATORE** e **CONDIZIONATORE**, a seconda della richiesta dell'utente. La periferica **VENTILATORE** ha a disposizione un registro di interfaccia di dimensione byte che consente di specificare la velocità di rotazione, secondo il seguente algoritmo:

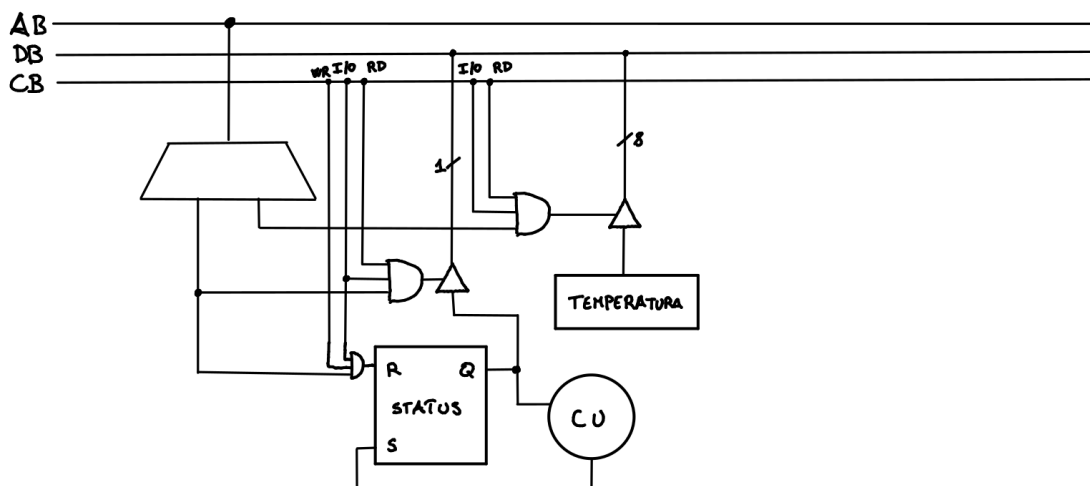
- se la differenza tra la temperatura attuale e quella richiesta è $< 4^{\circ}\text{C}$, il ventilatore viene attivato a velocità 1;
- se la differenza tra la temperatura attuale e quella richiesta è compresa tra 4°C e 8°C , viene attivato a velocità 2;
- altrimenti viene attivato a velocità 3.

Realizzare:

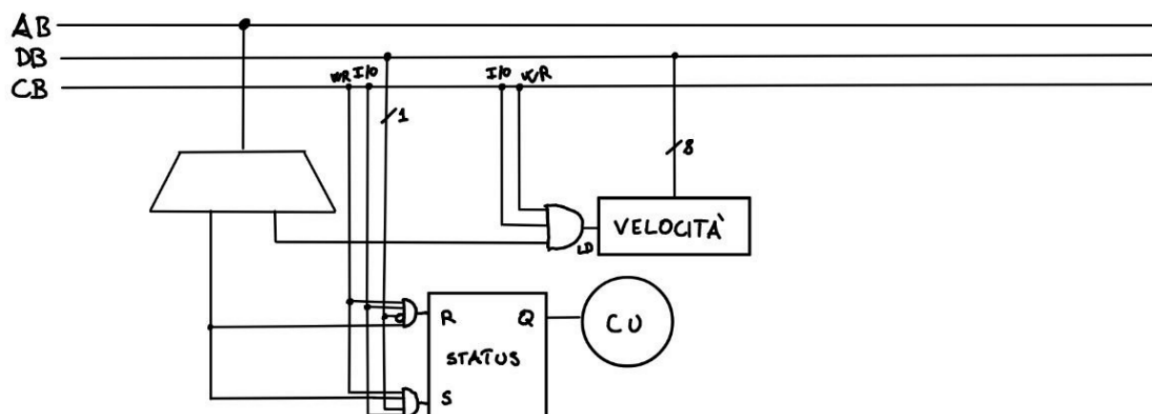
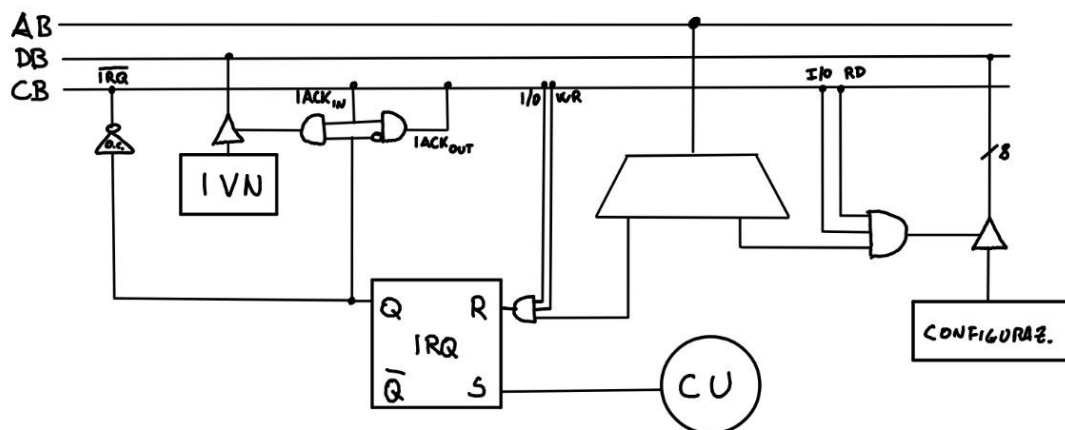
- Le interfacce delle periferiche **TERMOMETRO**, **RICEVITORE**, **VENTILATORE** (NON è necessario realizzare le interfacce delle periferiche **DEUMIDIFICATORE** e **CONDIZIONATORE**);
- Tutto il software necessario al funzionamento del sistema, comprensivo di eventuali driver.

Soluzione

Si presenta una delle tantissime possibili soluzioni. La periferica **TERMOMETRO** è una periferica sincrona di input, che fornisce dati ad 8 bit. L'interfaccia è la seguente:



La periferica **RICEVITORE** è asincrona di input. Poiché non è necessario mascherare le interruzioni di questa periferica (ossia, essa può generare in qualsiasi istante una richiesta di interruzione), non è necessario inserire un flip flop di STATUS. Tuttavia, la CU riceve anche il valore del flip flop IRQ, per evitare di sovrascrivere i dati di configurazione prima che il processore abbia servito una richiesta di interruzione precedentemente sollevata.



La funzione `calcola_velocita` prende un input nel registro `%dil` (che è stato preparato da un'altra funzione), lo elabora e produce un output nel registro `%al`. L'output sarà usato come parametro per impostare la velocità di un ventilatore. La logica implementata è la seguente:

1. Inizialmente, la funzione sottrae `%sil` (temperatura richiesta) da `%dil` (temperatura attuale) e memorizza il risultato in `%dil`. Quindi, `%dil` ora rappresenta la differenza tra la temperatura attuale e quella richiesta.
2. Viene inizializzata la velocità a 1 con `movb $1, %al`.
3. Successivamente, viene controllato se la differenza è inferiore o uguale a 3 (che significa che la differenza è inferiore a 4°C). Se lo è, la funzione restituisce immediatamente la velocità 1.
4. Se la differenza non è inferiore o uguale a 3, viene sottratto 4 da `%dil` e la velocità viene impostata a 0.
5. Viene poi controllato se la differenza aggiustata è inferiore o uguale a 4 (ossia, se la differenza originale era tra 4°C e 8°C). Se lo è, viene aggiunto 1 alla velocità, portandola a 1, e successivamente viene aggiunto un altro 1, portando la velocità a 2.

6. Se la differenza non è inferiore o uguale a 4, il codice salta l'aggiunta di 1, e aggiunge solo 2, portando la velocità a 2.
7. Infine, se la differenza originale era maggiore o uguale a 8, nessuno dei salti condizionali avrebbe avuto luogo, e la velocità sarebbe stata portata a 3.

Quindi sì, la funzione implementa correttamente l'algoritmo descritto.

La funzione `ferma_dispositivi` spegne tutti i dispositivi mandando zero ai loro registri di stato. Le funzioni `attiva_ventilatore`, `attiva_deumidificatore` e `attiva_condizionatore` attivano i rispettivi dispositivi inviando un segnale di accensione ai loro registri di stato. La funzione `attiva_ventilatore` inoltre legge un valore da un registro `TERM_STATUS`, elabora il valore attraverso la funzione `calcola_velocita`, e invia il risultato al registro `VELOCITA`.

Il codice del driver, quando viene servita la richiesta di interruzione, ferma tutti i dispositivi, legge un valore da un registro di configurazione, e attiva i dispositivi specificati in base al valore di configurazione. Al termine della routine, ripristina il registro `%al` e torna alla posizione nel codice da cui era stato interrotto.

```
1  .org 0x800
2  .data
3      .equ VENT_STATUS, 0x1
4      .equ DEUM_STATUS, 0x2
5      .equ COND_STATUS, 0x3
6      .equ TERM_STATUS, 0x4
7      .equ CONFIGURAZIONE, 0x5
8      .equ VELOCITA, 0x6
9      .equ RIC_IRQ, 0x7
10     .equ TEMPERATURA, 0x8
11  .text
12  main:
13      sti
14      .loop:
15      hlt
16      jmp .loop
17
18  calcola_velocita:
19      subb %sil, %dil
20      movb $1, %al
21      cmpb $3, %dil
22      jbe .ret
23      subb $4, %dil
24      xorb %al, %al
25      cmpb $4, %dil
26      jna .skip
27      addb $1, %al
28  .skip:
29      addb $2, %al
30  .ret:
31      ret
32
33  ferma_dispositivi:
34      # Assumo che tutti i dispositivi si comportino come VENTILATORE
35      # per quanto riguarda l'accensione e lo spegnimento
36      xorb %al, %al
37      outb %al, $VENT_STATUS
38      outb %al, $DEUM_STATUS
39      outb %al, $COND_STATUS
40      ret
41
42  attiva_ventilatore:
43      outb %al, $TERM_STATUS
```

```

44     .bw:
45     inb $TERM_STATUS, %al
46     btb $0, %al
47     jnc .bw
48     inb $TEMPERATURA, %al
49     movb %dil, %sil
50     andb $31, %sil
51     movb %al, %dil
52     call calcola_velocita
53     outb %al, $VELOCITA
54     movb $1, %al
55     outb %al, $VENT_STATUS
56     ret
57
58     attiva_deumidificatore:
59     movb $1, %al
60     outb %al, $DEUM_STATUS
61     ret
62
63     attiva_condizionatore:
64     movb $1, %al
65     outb %al, $COND_STATUS
66     ret
67
68     .driver 0 # RICEVITORE
69     call ferma_dispositivi
70     inb $CONFIGURAZIONE, %al
71     pushb %al # Copia locale della configurazione
72     outb %al, $RIC_IRQ
73     btb $7, %al
74     jnc .else1
75     movb %al, %dil # La configurazione ha anche la velocità
76     call attiva_ventilatore
77     jmp .end
78     .else1:
79     btb $6, (%rsp) # %al è stato sporcato dalle funzioni chiamate
80     jnc .else2
81     call attiva_deumidificatore
82     jmp .end
83     .else2:
84     btb $5, (%rsp)
85     jnc .end
86     call attiva_condizionatore
87     .end:
88     addq $1, %rsp
89     iret

```

Prova di laboratorio

Si realizzi un programma in C che, data una parola in input, generi tutte le permutazioni dei caratteri che compongono la parola e le salvi su file, una per riga. Il programma accetta due argomenti: la parola di cui utilizzare i caratteri per generare le permutazioni e il percorso al file in cui memorizzare il risultato della computazione.

Ad esempio, la seguente invocazione:

```
1 | ./permutazioni ROMA risultato.txt
```

crea il file `risultato.txt` (se non già esistente) con, ad esempio, il contenuto:

```
1 ROMA
2 ROAM
3 RMOA
4 RMAO
5 RAOM
6 RAMO
7 ORMA
8 ORAM
9 OMRA
10 OMAR
11 OARM
12 OAMR
13 MROA
14 MRAO
15 MORA
16 MOAR
17 MARO
18 MAOR
19 AROM
20 ARMO
21 AORM
22 AOMR
23 AMRO
24 AMOR
```

Si noti che se il file specificato esiste, il suo eventuale contenuto viene sovrascritto dal programma.

Soluzione

Il cuore dell'esercizio riguarda la generazione delle permutazioni. Ci sono diverse strategie che possiamo utilizzare per generare tutte le possibili permutazioni di una stringa:

1. **Ricorsione:** Questo è l'approccio più comune per risolvere questo tipo di problema. L'idea di base della ricorsione è dividere il problema in sotto-problemi più piccoli fino a raggiungere un caso base che può essere risolto facilmente. Nel contesto della generazione di permutazioni, possiamo fissare un carattere alla volta e permutare il resto della stringa. Dopo aver generato tutte le permutazioni con quel carattere fissato, passiamo al carattere successivo. Questo procedimento viene ripetuto ricorsivamente.
2. **Backtracking:** Il backtracking è una strategia simile alla ricorsione, ma con un passaggio in più: annulliamo alcune delle decisioni prese. Questo ci consente di esplorare diverse parti dell'albero delle soluzioni senza dover iniziare da capo ogni volta. Nel contesto della generazione di permutazioni, una volta generata una permutazione, possiamo "tornare indietro" e cambiare l'ultimo carattere per generare una nuova permutazione.
3. **Algoritmo di Heap:** Questo è un algoritmo specifico per generare permutazioni. È più efficiente dei metodi basati su ricorsione e backtracking perché genera ciascuna permutazione da quella precedente con un numero minimo di scambi. Questo lo rende particolarmente utile quando si lavora con stringhe molto lunghe.

È importante notare che, indipendentemente dalla strategia scelta, il numero di permutazioni di una stringa di lunghezza n è $n!$, quindi il tempo di esecuzione di qualsiasi algoritmo per generare tutte le permutazioni sarà almeno $O(n!)$. Pertanto, generare tutte le permutazioni di stringhe molto lunghe può essere molto dispendioso in termini di tempo.

La soluzione proposta è la seguente:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  void permutazioni(char *stringa, char *s, size_t livello, FILE *out_file)
6  {
7      size_t len_s = strlen(s);
8      size_t len_radice = stringa ? strlen(stringa) : 0;
9
10     for (int i = 0; i < len_s; i++) {
11         char t[BUFSIZ];
12         char new_s[BUFSIZ];
13
14         if (stringa) {
15             strncpy(t, stringa, len_radice);
16         }
17         t[len_radice] = s[i];
18         t[len_radice + 1] = '\0';
19
20         if (strlen(t) == livello)
21             fprintf(out_file, "%s\n", t);
22
23         strncpy(new_s, s, BUFSIZ - 1);
24         new_s[BUFSIZ - 1] = '\0'; // Ensure null-termination
25         memmove(new_s + i, new_s + i + 1, len_s - i);
26         permutazioni(t, new_s, livello, out_file);
27     }
28 }
29
30 int main(int argc, char *argv[])
31 {
32     char my_string[BUFSIZ];
33
34     if (argc != 3) {
35         printf("Please provide exactly two arguments: word and file path\n");
36         exit(EXIT_FAILURE);
37     }
38
39     strncpy(my_string, argv[1], BUFSIZ - 1);
40     my_string[BUFSIZ - 1] = '\0'; // Ensure null-termination
41
42     FILE *out_file = fopen(argv[2], "w");
43     if (out_file == NULL) {
44         printf("Could not open file for writing: %s\n", argv[2]);
45         exit(EXIT_FAILURE);
46     }
47
48     permutazioni(NULL, my_string, strlen(my_string), out_file);
49
50     fclose(out_file);
51
52     return 0;
53 }

```

Il cuore della soluzione risiede nella funzione `permutazioni`. La funzione `permutazioni` genera tutte le permutazioni possibili di una data stringa utilizzando un approccio ricorsivo e scrivendo ogni permutazione in un file specificato. In termini di argomenti, accetta due stringhe (`stringa` e `s`), una dimensione (`livello`, che rappresenta la lunghezza della stringa originale) e un puntatore a un file (`out_file`).

All'inizio della sua esecuzione, la funzione calcola le lunghezze delle due stringhe fornite. Si noti che `stringa` può essere `NULL` nella chiamata iniziale, e in tal caso la sua lunghezza viene considerata come 0.

Una volta ottenute le lunghezze, la funzione entra in un ciclo, iterando su ogni carattere della stringa `s`. In ogni iterazione del ciclo, crea due nuove stringhe, `t` e `new_s`. La stringa `t` viene creata come copia di `stringa`, con l'aggiunta del carattere corrente di `s` alla fine. Se `stringa` è `NULL`, `t` sarà composta solo dal carattere corrente di `s`. D'altra parte, `new_s` è una versione di `s` con il carattere corrente rimosso. Questa stringa è ottenuta usando `memmove` per spostare tutti i caratteri successivi indietro di una posizione.

Dopo aver creato `t` e `new_s`, la funzione controlla se la lunghezza di `t` è uguale a `livello`, che è la lunghezza della stringa originale. Se lo è, `t` è una permutazione completa e viene quindi scritta nel file specificato.

L'ultimo passo nel corpo del ciclo è una chiamata ricorsiva a `permutazioni`, passando `t` come nuova radice e `new_s` come nuova stringa su cui eseguire le permutazioni.

La funzione continua a eseguire queste operazioni, costruendo gradualmente permutazioni, rimuovendo caratteri e ricorrendo sulle stringhe rimanenti fino a quando tutte le possibili permutazioni sono state generate e scritte nel file.

Una soluzione alternativa può essere la seguente:

```
1 void swap(char *x, char *y)
2 {
3     char temp;
4     temp = *x;
5     *x = *y;
6     *y = temp;
7 }
8
9 void permutazioni(char *str, int l, int r, FILE *out_file)
10 {
11     int i;
12     if (l == r)
13         fprintf(out_file, "%s\n", str);
14     else {
15         for (i = l; i <= r; i++) {
16             swap((str+l), (str+i));
17             permute(str, l+1, r);
18             swap((str+l), (str+i)); // backtrack
19         }
20     }
21 }
```

In questo caso, viene effettuato lo swap esplicito dei caratteri all'interno di un ciclo, per esplorare tutte le possibili soluzioni in maniera ricorsiva. Se si vuole evitare la chiamata ricorsiva, è possibile utilizzare l'algoritmo di Heap:

```
1 void permutazioni(char *str, int size, FILE *out_file)
2 {
3     // Questo array tiene traccia della posizione in cui
4     // stiamo permutando ad ogni livello equivalente di ricorsione.
5     int *indices = (int *)calloc(size, sizeof(int));
6
7     // L'array di indici inizialmente contiene tutti zero.
8     fprintf(out_file, "%s\n", str);
9
10    // Fintanto che non abbiamo esaurito tutte le permutazioni...
11    int i = 0;
12    while (i < size) {
13        if (indices[i] < i) {
14            // ... scambia il carattere corrente se `i` è dispari con il primo carattere...
15            // ... altrimenti scambia con il carattere ad indice `indices[i]`.
16            swap(&str[i % 2 == 1 ? indices[i] : 0], &str[i]);
```

```
17
18         fprintf(out_file, "%s\n", str);
19
20         // Incrementa il contatore per la posizione corrente e azzeri i contatori per tutte le
posizioni di cui hai già calcolato le permutazioni.
21         indices[i]++;
22         i = 0;
23     } else {
24         // Se abbiamo esaurito tutte le permutazioni per questa posizione, torna indietro.
25         indices[i] = 0;
26         i++;
27     }
28 }
29
30 free(indices);
31 }
```